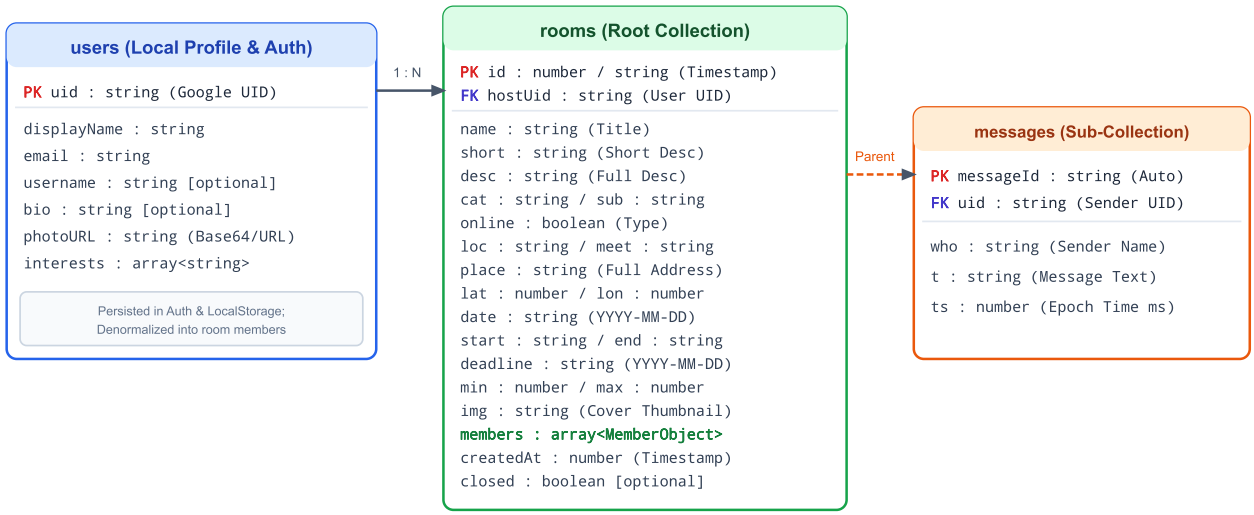
NUDTEE Data Model & Database Schema

Google Cloud Firestore Data Architecture (NoSQL Document Store)

Data Architecture (NoSQL Firestore Schema): Designed as a Document-Oriented Database optimized for real-time listener subscriptions (FR-3) and Atomic Transactions (FR-2) to prevent race conditions. `rooms` serves as the primary Root Collection, utilizing a sub-collection `messages` for in-room chats (FR-5), paired with LocalStorage for storing full user profiles.

Physical Entity-Relationship Diagram (Firestore Collections)



Root Collection Schema: `rooms`

Field Name	Data Type	Nullability	Mapped FR	Description / Constraints
`id`	Number / String	**Required** PK	FR-1	Unique session room identifier generated from creation timestamp + random suffix.
`name`	String	**Required**	FR-1	Title of the activity session (Max 60 characters).
`hostUid` / `host`	String	**Required** FK	FR-1	Google Account UID of the Organizer and snapshot of their display name.
`cat` / `sub`	String	**Required**	FR-1	Main category (21 categories) and sub-category for session filtering.
`online`	Boolean	**Required**	FR-1, FR-4	`true` = Online session (no map), `false` = Offline session (with map pin).
`loc` / `meet` / `place`	String	**Required**	FR-1, FR-4	Short location (loc), meeting point / online link (meet), and full geocoded address (place).
`lat` / `lon`	Number (Float)	Optional	FR-4	Latitude & Longitude coordinates from OpenStreetMap/GPS for interactive map placement.
`date` / `start` / `end`	String	**Required**	FR-1, FR-6, FR-7	Event date (YYYY-MM-DD), start and end times (HH:mm) for overlap checks and calendar export.
`deadline`	String	**Required**	FR-1, FR-6	Registration deadline date (YYYY-MM-DD) used for automated session closing.
`min` / `max`	Number (Integer)	**Required**	FR-1, FR-2	Minimum headcount (min ≥ 1) and maximum capacity limit (max ≥ min).

Field Name	Data Type	Nullability	Mapped FR	Description / Constraints
`members`	Array<Object>	Required Array	FR-1, FR-2	List of joined participants (Denormalized Objects: uid, name, photo, bio, etc.).
`img`	String (Base64)	Optional	FR-1	Canvas-resized Base64 Data URL for the room cover image.

Sub-Collection Schema: `rooms/{roomId}/messages` (FR-5 In-Room Chat)

Stores real-time chat messages under each room document to isolate chat traffic, optimize real-time snapshot listeners, and enforce access controls.

Field Name	Data Type	Nullability	Mapped NFR	Description / Constraints
`messageId`	String	Required PK	NFR-1	Auto-generated Firestore Document ID via `addDoc()`.
`uid`	String	Required FK	NFR-5	Google Account UID of the sender (must exist in room's `members` array).
`who`	String	Required	NFR-1	Snapshot of the sender's display name at the time of sending.
`t`	String	Required	FR-5, NFR-1	The text content of the message.
`ts`	Number (Epoch ms)	Required	FR-5, NFR-1	Epoch timestamp in milliseconds used for chronological ordering (`orderBy('ts')`).

Embedded Object Structure: `MemberObject` (Denormalized)

To operate strictly within free-tier Spark quota limits and minimize document reads, participant data is denormalized directly inside the room's `members` array.

Property Name	Data Type	Nullability	Description & Privacy Rules
`uid`	String	Required	Google Account ID of the participant. Used for authentication and authorization.
`name`	String	Required	Display name configured in the user's profile.
`user`	String	Optional	Username handle (e.g., @john_doe) for participant card displays.
`photo`	String	Optional	Small thumbnail image (Canvas-resized 96x96px Base64 Data URL).
`bio` / `interests`	String / Array	Optional	Short bio and interest tags displayed in participant profile modals.

Data Integrity Constraints & Security Logic

1. Race Condition Prevention (FR-2 & NFR-4)
Join and Leave operations execute via **Cloud Firestore Atomic Transactions** (`runTransaction`). The transaction re-reads the latest participant count before updating. If `members.length >= max`, the transaction is rejected instantly, guaranteeing that remaining spots never go negative.

2. Real-time Status Evaluation (FR-3 & FR-6)
Session statuses (Open, Almost Full, Full, Joined, Closed) are dynamically evaluated on client devices and real-time listeners:
• `Closed`: Evaluated when `currentTime > deadline(23:59)` or `currentTime >= startTime`
• `Full`: Evaluated when `current_joined == max_players`

3. In-Room Chat Security (FR-5 & NFR-5)
Access control to the `messages` sub-collection is strictly enforced by Security Rules: `request.auth.uid in get(.../rooms/roomId).data.members[*].uid` This ensures non-participants cannot read or send messages in unjoined rooms.

4. Time Overlap Validation (FR-1 & FR-2)
Prior to creating or joining a session, the client checks the `[start, end]` timeframe against all active sessions already joined by the user. If an overlap is detected, the action is rejected with the name of the conflicting session.